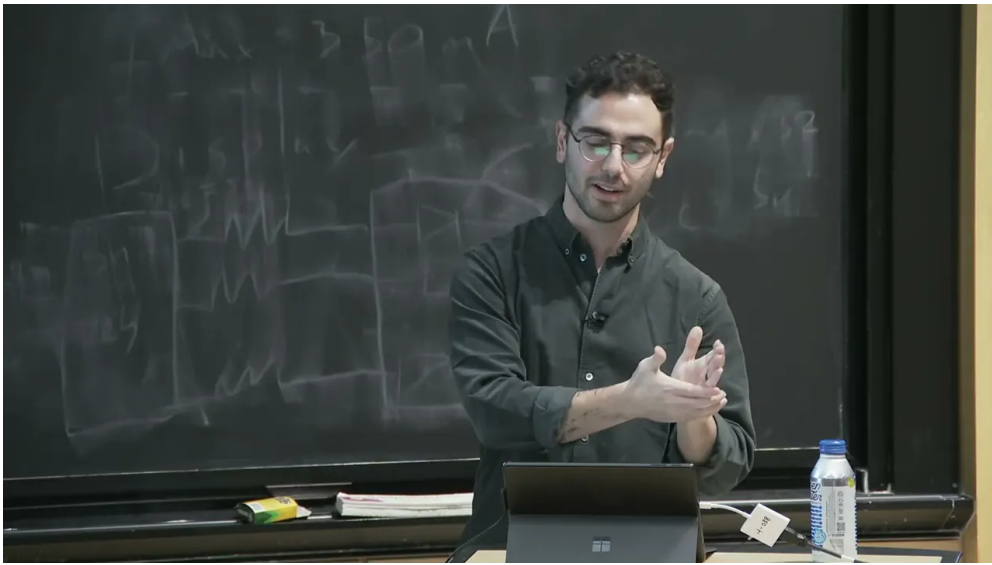


课程笔记

课程笔记：课程结构与 Julia 入门

Codex

2026-04-15



视频作者/频道：MIT Video Productions

发布日期：2025-01-13

视频时长：27:26

视频链接：<https://www.youtube.com/watch?v=ekwu47RHCHE>

目录

1 课程开场与安装准备	2
1.1 本章小结	2
2 本周安排与学习方式	2
2.1 本章小结	3
3 Julia 的基本定位	3
3.1 本章小结	3
4 交互式开发: REPL、VS Code 与 Jupyter	4
4.1 本章小结	4
5 表达力: 单位、循环、宏与 Neuroblox	4
5.1 本章小结	5
6 类型系统、多重派发与自定义结构	5
6.1 本章小结	5
7 性能、社区与学习资源	5
7.1 本章小结	6
8 Notebook 实操与课程收尾	6
8.1 本章小结	6
9 总结与延伸	6
9.1 本章小结	7

1 课程开场与安装准备

这一讲一开始并不是直接讲 Julia 语法，而是先把课堂运行环境理顺。讲者先把大家带到课程主页，提醒同学们先确认已经完成 Julia、Neuroblox 以及后续教程所需依赖的安装。这里的关键不是“装过就行”，而是要把课程的项目环境和依赖锁定到当前版本，避免后面练习时因为旧包版本而出问题。

```
1 using Pkg
2 using Downloads
3
4 Downloads.download(
5     "https://raw.githubusercontent.com/Neuroblox/course/refs/heads/main/Project.
6         toml",
7     joinpath(@__DIR__, "Project.toml")
8 )
9
10 pkg"registry add General"
11 Pkg.instantiate()
```

上面这段安装流程的逻辑很简单：先把课程提供的 `Project.toml` 下载到本地课题目录，再添加 Julia 的通用 registry，最后用 `Pkg.instantiate()` 生成与课程匹配的 `Manifest.toml` 和依赖环境。讲者特别强调，如果你在今天之前已经跑过这套步骤，最好把旧的 `Project.toml` 和 `Manifest.toml` 删掉后重跑一次，因为 Neuroblox 和其依赖最近有过更新。

安装时最容易出错的地方

课程要求你每次打开 notebook 时，都在包含 `Project.toml` 与 `Manifest.toml` 的目录下启动 Julia/Jupyter。这样做的目的不是繁琐，而是让每个人都在同一套依赖下运行教程。

1.1 本章小结

这一部分的重点不是 Julia 本身，而是先把“课程计算环境”这件事做对：统一项目目录、重新生成依赖、把 notebook 和项目文件放在同一套可复现的环境里。

2 本周安排与学习方式

讲者随后把整周安排快速过了一遍，并明确了这门课的节奏：每个 session 先有 10 到 15 分钟的概念性介绍，其余时间主要留给 notebook、文本块和代码块。换句话说，这不是一门以长讲座为中心的课，而是一门以动手练习为中心的课。

时间	内容
今天	课程结构 + Julia 入门
明天上午	Julia 中的微分方程与绘图生态（讲者口头写作 <i>Mikey</i> ，应为 Julia 绘图生态）
明天下午	Neuroblox 核心：blocks、connections、如何搭建自己的模块
周三	神经元、神经质量、外部来源，以及 circuit models
周四	decision making 与 reinforcement learning / synaptic plasticity
周五上午	spectral DCM 与模型拟合
周五下午	开放式工作时间：练习、挑战题、提问、继续搭建自己的 circuits

这一整套安排里有两个信息很重要。第一，课程的后续主题都会尽量变成“可运行的 notebook 任务”，而不是只停留在讲解层面。第二，每天的 notebook 末尾都会带有 challenge problems，它们不是额外负担，而是“学完这一部分后还能继续往前走”的建议方向。

课程的工作流

先看网页上的内容，再到 notebook 里跑同样的材料；完成基础练习后，再做 challenge problems。周五的最后一场则故意保持开放，方便大家回头补练习、问问题，或者开始建模自己的电路。

2.1 本章小结

这门课的设计目标很明确：前面用极短的讲解建立框架，后面把时间留给动手。课程安排、练习和 challenge problems 共同组成了同一条学习路径，而不是彼此割裂的任务清单。

3 Julia 的基本定位

讲者接着把话题切到 Julia 本身。Julia 从 2009 年在 MIT 开始开发，直到 2018 年 8 月才发布 1.0。这个时间线的意义在于，它说明 Julia 是一门较年轻但成长很快的语言；而今天的定位，则是“动态、交互式、表达力强，同时又能接近 C 的性能”。

讲者给出的 Julia 画像

Julia 不是“只能写高层表达”的脚本语言，也不是“只能跑得快但很难写”的底层语言。它试图同时兼顾可读性、交互性和性能，这也是“*like Python, runs like C*”这句口号的真实含义。

讲者还特别先提醒了一个会立刻影响上手体验的规则：Julia 是 **one-based indexing**，也就是任何集合的第一个元素索引都是 1，而不是 Python 里的 0。对于从 Python 迁移过来的同学，这一点需要刻意适应。

3.1 本章小结

Julia 的核心卖点可以概括为三层：语法高层、交互友好、性能可观。课程后面所有关于 Neuroblox 的设计，基本都建立在这三个性质之上。

4 交互式开发：REPL、VS Code 与 Jupyter

讲者进一步解释“dynamic and interactive”到底是什么意思。Julia 虽然是编译型语言，但在日常使用中可以像 Python 或 MATLAB 那样按行执行、即时看结果。课程把这种体验放在三个入口里讲：REPL、VS Code 和 Jupyter Notebook。

```
julia --project="." -e "using IJulia; notebook(dir = pwd());"
```

REPL 是最直接的方式，适合快速试验、验证一个表达式是否正确；VS Code 则更适合更长一点的代码开发，因为它能提供补全、文档和绘图面板。讲者专门展示了 Julia 扩展如何在你输入函数名时给出补全，并在旁边显示 `docstring`。如果你在 session 里画图，VS Code 还会打开图窗并保留历史图像，便于回看。

而本课最主要的工作界面是 Jupyter Notebook。讲者强调，它与网页课程内容是同一份材料，只是呈现形式略有不同：网页上是按页面排版的文本与代码，而 notebook 则是文本块和代码块交错的可执行版本。也正因为如此，课程建议大家以 notebook 为主，而不是只看网页。

关于 notebook 的使用建议

讲者建议：如果你第一次打开 Julia intro notebook，先完成前面的基础内容；性能那一节内容很长，短时间内可以先略过，后面有空再回来补。

4.1 本章小结

Julia 的“交互式”不是说它不是编译型语言，而是说它允许你在编译性能和即时反馈之间取得平衡。REPL 适合试错，VS Code 适合开发，Jupyter 适合课程式学习和循序渐进地跑代码。

5 表达力：单位、循环、宏与 Neuroblox

讲者接下来证明 Julia 不只是“能交互”，还“很会表达”。最直观的例子是 `Unitful` 这类包：你可以像写论文一样，把物理单位直接嵌进代码里，例如电阻写成 `100 megaohms` 之类的形式。这样做的好处是，代码更接近人类描述，减少“在脑子里换算单位”的负担。

另一点非常重要：Julia 并不强迫你把循环改写成向量化表达式。讲者明确说，在 Julia 里，循环和向量化操作的性能可以非常接近，因此你可以优先考虑可读性，而不是被某种“必须向量化”的习惯绑定。

更进一步，Julia 允许你通过 `macros` 搭建自己的领域专用语法。讲者在这里举了 `ModelingToolkit` 的例子：用 `@variables`、`@parameters`、`@equations` 这类宏写出符号化的微分方程系统。这样，代码看上去更像数学推导，而不是底层实现。

同样的思路也出现在 `JuMP` 里。你先定义模型、选择优化器，然后再逐步添加变量、目标函数和约束，最后打印或者求解模型。讲者用它说明：Julia 不是只适合“写语法”，它也适合把复杂优化问题写成清晰的高层建模语句。

在 `Neuroblox` 中，这种表达力更直接地服务于建模。讲者展示了一个非常短的例子：定义两个 Hodgkin-Huxley 神经元，创建一个空图，然后把抑制性神经元连到兴奋性神经元上。也就是说，电路模型在代码层面就是图结构上的节点和边，基本概念一一对应。

为什么这对 Neuroblox 特别重要

Neuroblox 的目标不是把你锁进一套复杂 API，而是让“神经元、连接、图、规则”这些建模对象能在 Julia 里自然组合。Julia 的宏、类型和高层语法正好给了它这种组合能力。

5.1 本章小结

这一部分的核心不是某个包，而是 Julia 的建模风格：代码可以既像数学，也像配置，还能保持可执行和可组合。对 Neuroblox 来说，这意味着电路建模可以写得短、清楚，而且容易扩展。

6 类型系统、多重派发与自定义结构

讲者把 Julia 的第二个核心概念讲得很清楚：**multiple dispatch**。简化地说，Julia 允许同一个函数名对应多个方法，而究竟调用哪个方法，不是看函数名字，而是看参数的组合和类型。这个机制是 Julia 设计里的底层支柱之一。

```
1 f(x::Float64, y::Float64) = ...  
2 f(x::Number, y::Number) = ...
```

讲者用一个非常短的例子说明派发规则：如果你传入的是 2.0 和 3，会匹配到更一般的 `Number` 方法；如果你传入的是两个浮点数，则会命中更具体的 `Float64` 方法。这里的重点在于，`Float64 <: Number` 这类 subtype 关系会直接影响方法选择。

Julia 的类型系统还允许你定义自己的结构体。讲者用一个 `MyType` 的例子说明：结构体里的某些字段可以指定具体类型，某些字段也可以保持泛型，这样既能约束数据，又能保留灵活性。这个机制在后续创建自定义 blocks 时会非常有用，因为 Neuroblox 的许多组件本质上就是类型明确、但仍可扩展的数据结构。

类型和多重派发组合在一起，会产生一种很 Julia 的现象：两个原本没打算互相配合的类型和函数，可能只需要一条方法定义，甚至不需要额外代码，就突然能一起工作。讲者把这种现象称为一种“emergent”式的语言能力。

这和面向对象的差别

讲者把 Julia 的设计与 Python 式面向对象做了对比：Julia 更强调“按参数类型组合方法”，而不是把所有行为都塞进对象层级里。对数值计算和科学建模来说，这通常更自然。

6.1 本章小结

类型系统和 multiple dispatch 不是 Julia 的附加功能，而是它的编程核心。理解这两点，后面看 Neuroblox 的自定义 block、连接规则和模型组织方式时，就会顺很多。

7 性能、社区与学习资源

讲者最后补了两块很实用的内容：一是性能，二是学习资源。性能部分的核心信息是，Julia 在很多基准上可以非常接近 C 的行为，而不像 MATLAB、R 或 Octave 那样离底层实现更远。讲者用线性代数和递归类基准说明：Julia 既能处理数值密集计算，也能在高层抽象下维持相当不错的速度。

除了语言本身，Julia 社区也被强调为一个优势。讲者提到 [Discourse.julialang.org](https://discourse.julialang.org)、Julia Slack，以及开发者调查里大家最常使用的学习资源。这里最值得记住的是：官方 manual 在 Julia 生态里并不是摆设，而是最常被拿来学习和查找细节的材料之一。

资源类型	课程建议
官方 manual	优先阅读；讲者明确建议把它当作首要参考
JuliaCon / YouTube / 书籍	适合补充背景、看范例、查不同作者的讲法
课程 Introduction to Julia 页面	已经整理了若干相关链接，适合继续扩展
社区论坛与 Slack	遇到具体问题时，先搜索再提问，通常能很快得到反馈

7.1 本章小结

性能说明 Julia 不是“写起来快但跑不动”，社区和资源则说明它不是“只有语言没有生态”。对于真正要在科研里持续使用的工具，这两点都很关键。

8 Notebook 实操与课程收尾

在概念讲完之后，讲者把大家拉回到实际操作：进入 Introduction to Julia 页面，打开第一个 notebook `julia_intro`。如果你是第一次用 Jupyter，就按正常流程进入 notebooks 文件夹，然后打开对应页面即可。后续每个 notebook 的进入方式都类似。

讲者还给了一个很实用的时间管理建议：`julia_intro` 这份 notebook 很长，但并不要求一次性全部做完。前面的核心部分，尤其是 `functions` 之后的内容，值得认真跟；但性能那一段可以先跳过，等你完成前面的内容再回来补。这样可以保证第一次上手时不被过长的 notebook 拖住。

最后，讲者再次强调：课程网页和 notebook 的内容是一致的，所以你可以在网页上预览，再到 notebook 里执行同一套材料。遇到问题时，既可以在 Slack 提问，也可以直接举手。

8.1 本章小结

这门课的实际执行方式非常明确：先用网页建立概念，再在 notebook 里跑同样的材料；先抓住主线内容，再回头处理性能和细节。这样做的目的，是让你尽快进入“边看边跑边改”的状态。

9 总结与延伸

这一讲的真正任务，不是完整教会 Julia，而是让你建立一个足够稳的起点：知道课程怎样运行，知道 notebook 怎么打开，知道 Julia 为什么适合 Neuroblox，也知道哪些概念会在后续反复出现。

如果把整讲压缩成三句话，就是：

- 先把环境和项目文件弄对，保证后续练习可复现；
- 再记住 Julia 的三个关键词：交互、表达力、性能；
- 最后把 `multiple dispatch`、类型和宏当成后续理解 Neuroblox 的基础设施。

对后续内容来说，今天的目标已经足够清楚：你不需要立刻掌握 Julia 的全部细节，但你必须能在 notebook 里开始跑，知道自己在看什么，也知道自己下一步该去哪里找答案。

9.1 本章小结

课程结构、Julia 语言特性和 notebook 工作流在这一讲里被连接起来了。后面的每一讲，都会把今天打下的这些基础继续展开到具体模型和具体电路上。